



Khulna University
Department of Computer Science and Engineering

FOIL and Beam Search Algorithm Implementation on Lecture Graph Dataset

Machine Learning Assignment Report

Submitted To	Professor Dr. G M Atiqur Rahaman
Course	Machine Learning
Topic	Learning First-Order Logic / Rule Learning
Department	Computer Science and Engineering
University	Khulna University
Student Name	Md. Farid Hossen Rehad
Student ID	220222
Submission	May 2026

Abstract

*This report presents the implementation of simplified FOIL (First-Order Inductive Learner) and Beam Search algorithms applied to a directed graph dataset sourced from the course lecture slides. The background predicate is **edge(X,Y)** and the target predicate is **path(X,Y)**. Python is employed to enumerate positive and negative examples, compute FOIL Gain, execute Beam Search, and generate structured output files. Prolog is used to represent the graph facts and validate the learned rules through logical queries. The algorithms successfully induce the following rules:*

```
path(X,Y) :- edge(X,Y).
```

```
path(X,Y) :- edge(X,Z), path(Z,Y).
```

1. Introduction

Rule learning is a supervised machine learning paradigm in which the output consists of human-readable logical rules rather than opaque model parameters. In first-order logic (FOL), these rules can express relational patterns using variables and predicates. For instance, *path(X,Y)* asserts that a directed path exists from node X to node Y within a graph.

The graph-path problem is an ideal benchmark for relational learning because its structure maps naturally to logical facts. Direct connections between nodes are encoded as *edge(X,Y)* facts, while reachability is captured by the target predicate *path(X,Y)*. Correctly learning this predicate requires both a base case (direct edges) and a recursive rule (multi-hop paths), making it a non-trivial inductive challenge.

2. Objectives

- Implement a simplified version of the FOIL rule-learning algorithm.
- Implement a General-to-Specific Beam Search algorithm with configurable beam width.
- Apply both algorithms to the directed graph dataset from the lecture slides.
- Learn logical path rules from edge facts using inductive reasoning.
- Verify the correctness of learned rules using SWI-Prolog.
- Analyse and compare the performance characteristics of FOIL and Beam Search.

3. Dataset Description

The dataset consists of a directed graph with six nodes. Background knowledge is represented as *edge(X,Y)* facts, and the target concept to be learned is *path(X,Y)*. Negative examples are generated via the **Closed-World Assumption (CWA)**: any ordered pair of nodes not listed as a positive example is treated as a negative example.

Graph Edges (Background Knowledge)

```
edge(1,2).   edge(1,3).   edge(3,6).
edge(4,2).   edge(4,6).   edge(6,5).
```

Positive Path Examples

```
path(1,2). path(1,3). path(1,6). path(1,5).
path(3,6). path(3,5).
path(4,2). path(4,6). path(4,5).
path(6,5).
```

The graph has nodes {1, 2, 3, 4, 5, 6}. Nodes 2 and 5 have no outgoing edges, making them sink nodes. All reachable pairs are included as positive examples (10 total), and the remaining 20 ordered pairs constitute the negative example set.

4. FOIL Algorithm

FOIL (First-Order Inductive Learner), introduced by Quinlan (1990), is a sequential covering algorithm that constructs first-order Horn clauses from labelled examples. Starting with the most general clause — an empty rule body — FOIL iteratively specialises the current clause by appending the literal that maximises *FOIL Gain* until no negative examples remain covered.

Procedure

```
Initialise: clause body = {} (most general rule)
While negatives are still covered by the current clause:
    Generate candidate literals: edge(X,Y), edge(X,Z), edge(Z,Y), path(Z,Y)
    Compute FOIL Gain for each candidate
    Select the literal L with maximum gain
    Append L to the clause body
Remove positives covered by the completed clause
Repeat from the beginning if positives remain
```

5. FOIL Gain

FOIL Gain quantifies the information gain obtained by adding a new literal to the current partial clause. A candidate literal with high gain retains many positive bindings while eliminating negative ones.

Formula

$$\text{Gain}(L, R) = t \times [\log_2(p_1 / (p_1 + n_1)) - \log_2(p_0 / (p_0 + n_0))]$$

Symbol	Description
p0	Positive bindings before adding literal L
n0	Negative bindings before adding literal L
p1	Positive bindings after adding literal L

n1	Negative bindings after adding literal L
t	Number of positive tuples still covered by the clause

6. Beam Search Algorithm

Beam Search is a heuristic best-first search strategy that generalises greedy search by maintaining a *beam* of the top-k candidate hypotheses at each refinement step. Unlike FOIL, which commits irrevocably to a single best literal, Beam Search explores a broader portion of the hypothesis space, reducing the risk of being trapped in a locally optimal but globally suboptimal clause.

In this implementation, beam width $k = 3$ is used. At each level, candidate clauses are generated by appending one literal to each beam member. All candidates are scored by accuracy and coverage, and only the top 3 are retained for the next iteration.

Key Parameters

- **Beam width $k = 3$:** number of candidate rules maintained at each refinement level.
- **Scoring metric:** composite of accuracy (precision) and coverage (recall).
- **Termination:** when no new literal improves the top beam candidate.

7. Implementation Details

Project Structure

```
FOIL_BeamSearch_Graph_Assignment/  
├── data/  
│   └── graph_dataset.py  
├── src/  
│   ├── dataset.py  
│   ├── foil.py  
│   ├── beam_search.py  
│   └── main.py  
├── prolog/  
│   └── graph_path_rules.pl  
├── output/  
│   ├── foil_output.txt  
│   ├── beam_search_output.txt  
│   └── prolog_query_output.txt  
├── documentation/  
│   └── assignment_report.md  
└── README.md
```

File Descriptions

File	Purpose
data/graph_dataset.py	Stores nodes, edges, and positive path examples
src/dataset.py	Loads data and generates negative examples (CWA)
src/foil.py	Implements simplified FOIL rule learner
src/beam_search.py	Implements General-to-Specific Beam Search
src/main.py	Orchestrates the full pipeline and writes outputs
prolog/graph_path_rules.pl	Contains Prolog facts and learned path rules
output/foil_output.txt	FOIL algorithm results
output/beam_search_output.txt	Beam Search results
output/prolog_query_output.txt	Prolog query verification output

8. Prolog Representation and Verification

SWI-Prolog is used to represent the learned rules and validate them against the dataset. The background facts encode the graph edges, and the two induced rules define the path predicate. Prolog's built-in depth-first search and unification mechanism automatically handles the recursive rule.

Facts and Rules

```
% Background knowledge
edge(1,2). edge(1,3). edge(3,6).
edge(4,2). edge(4,6). edge(6,5).

% Learned rules
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

Verified Queries

```
?- edge(1,2).      % true  - direct edge
?- path(1,5).      % true  - via 1->3->6->5
?- path(4,5).      % true  - via 4->6->5
?- path(2,5).      % false - node 2 is a sink
```

9. Output and Result Analysis

FOIL Output (Condensed)

FOIL Learning Started

Target Predicate: path(X,Y)

Iteration 1:

Candidate: edge(X,Y) [highest gain]

Learned Rule 1: path(X,Y) :- edge(X,Y).

Iteration 2:

Candidates: edge(X,Z), path(Z,Y)

Learned Rule 2: path(X,Y) :- edge(X,Z), path(Z,Y).

All positives covered. Learning complete.

Beam Search Output (Condensed)

Beam Search Initialised | Beam width: 3

Level 1 beam:

[1] path(X,Y) :- edge(X,Y). score: 0.91

[2] path(X,Y) :- edge(X,Z). score: 0.76

[3] path(X,Y) :- edge(Z,Y). score: 0.71

Level 2 beam (from candidate [1]):

[1] path(X,Y) :- edge(X,Z),path(Z,Y). score: 1.00

Final Rules:

path(X,Y) :- edge(X,Y).

path(X,Y) :- edge(X,Z), path(Z,Y).

Both algorithms converge on the same two-rule hypothesis. Rule 1 handles direct one-hop paths; Rule 2 recursively handles arbitrary-length paths. Together, they achieve 100% coverage of the ten positive examples with zero false positives.

10. Comparison: FOIL vs. Beam Search

Criteria	FOIL	Beam Search
Search Type	General-to-Specific	General-to-Specific
Selection Method	FOIL Gain (greedy)	Best k candidates
Memory Usage	Lower	Higher (beam width)

Search Strategy	Single best literal	Multiple candidates
Result Quality	Good (may miss opt.)	Often better
Interpretability	High	High
Scalability	Moderate	Moderate

On this small dataset both methods arrive at identical rules. The advantage of Beam Search becomes significant on larger, noisier datasets where greedy literal selection in FOIL can lead to suboptimal clauses. The higher memory footprint of Beam Search is the trade-off for its broader exploration of the hypothesis space.

11. Limitations

- The FOIL implementation is simplified and does not handle negated literals or function symbols.
- The dataset is small (6 nodes, 6 edges); results may not generalise to larger graphs.
- Beam width is fixed at 3; sensitivity analysis across different widths was not conducted.
- Recursive rule induction in FOIL relies on heuristic ordering of candidate literals.
- Negative examples are generated solely by CWA, which may not reflect real-world distributions.

12. Conclusion

This assignment demonstrated the practical application of two foundational rule-learning algorithms — FOIL and Beam Search — to a directed graph dataset. Both algorithms successfully induced a two-clause hypothesis that correctly captures the transitive reachability relation:

```
path(X,Y) :- edge(X,Y).
```

```
path(X,Y) :- edge(X,Z), path(Z,Y).
```

Prolog verification confirmed that the learned rules correctly evaluate `path(1,5)` and `path(4,5)` as true, and `path(2,5)` as false, consistent with the graph structure. The exercise highlights how first-order rule learning enables the discovery of recursive, human-interpretable patterns from relational data — a capability beyond the reach of propositional learners.

References

- [1] Quinlan, J. R. (1990). Learning Logical Definitions from Relations. *Machine Learning*, 5(3), 239–266.
- [2] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- [3] Course Lecture Slides: Learning First-Order Logic / Rule Learning, Machine Learning, Khulna University.
- [4] SWI-Prolog Reference Manual. <https://www.swi-prolog.org/pldoc/>
- [5] Lavrac, N., & Dzeroski, S. (1994). *Inductive Logic Programming: Techniques and Applications*. Ellis Horwood.